

Section 8: Join Operations

결국 row들을 가져와서 어떻게 join할 것인가?

→ 각 row source를 어떤 access path로 가져오고, 그 row source들을 어떤 join order와 join method로 결합할 것인가

Join 방식

loop join

- 가장 직관적인 방식
- driving table row 가져오고 그에 맞는 inner row 찾는 방식

```
from employee a join department b
on a.department_id = b.department_id
where b.department_id = 60;
```

→ b table unique 스캔해서 row 가져오고, 그에 맞는 department_id 를 a 테이블에서 가져오면 이것도 loop join 방식

| 무조건 이중포문 구조가 아니다.

merge sort join

- 정렬한 값 기준으로 join

조인 키 기준으로 양쪽 row source가 정렬되어 있으면 유리

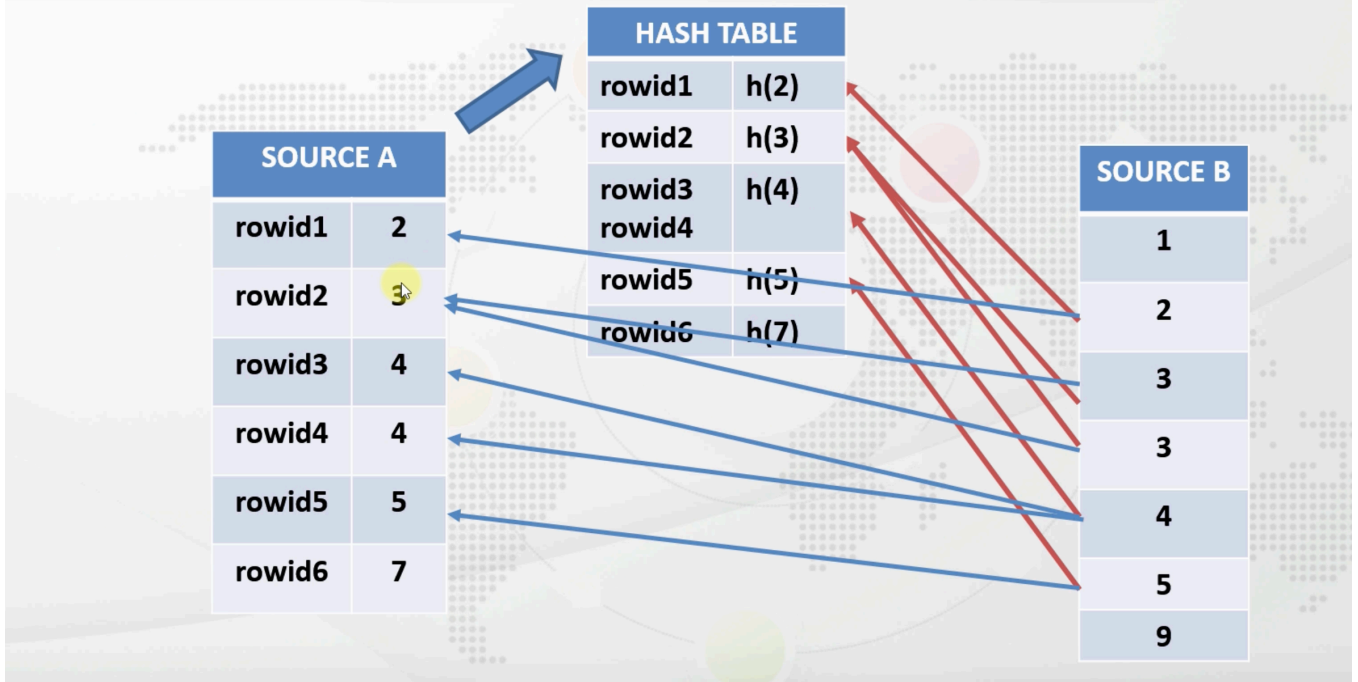
그 정렬 상태를

- 실제 sort 연산으로 만들 수도 있고
- index range/full scan으로 이미 정렬된 것을 활용할 수도 있음 → 유리

hash join

- row 작은 쪽을 해시테이블로 만들어서, 해시값 통해 다른쪽 row들의 key값 들과 join

HASH JOIN



- 작은 row source를 해시 테이블로 만든다
- 다른 row source를 읽으면서 조인 컬럼 값도 해시하여 대조한다
- 정렬을 이용하지 않는다
- 대용량 equijoin에서 강력하다

Join type

equi joins, none equi joins, outer joins, semi joins, anti joins

들이 있는데, 이 개념들이 중요하다기 보다,

db가 어떤 논리로 두 rows들을 join할지 판단하는 과정으로 이해함.
판단되면 hash, sorted, loop 등의 알고리즘으로 결합하는 것

Join Type: 어떤 논리 조건으로 결합할지

Join Method: 그걸 어떤 알고리즘으로 수행할지

기억할만한 내용은 exist 같은 서브쿼리 구문을 사용할 때도, 내부적으로 join 기법이 쓰인다는 것

```
select * from departments d
where exists (
select 1 from students s where d.department_id = s.department_id )
```

→ 학생이 1명이라도 있는 과를 고르는 것. 이 때 결국 student 테이블에서 해당 department_id가 하나라도 있으면 바로 ok

중복 매칭이 있어도 첫 매칭만 찾으면 멈춘다

해당쿼리 실행계획 예시

department 들을 가져오고, student의 department_id 인덱스를 range_scan하여 하나라도 있으면 ok

추가로, 서브쿼리는 논리적으로 동등하다면 다른 쿼리로 optimizer가 변형하여 실행될 수도 있다.